



Esercitazione 1

Introduzione a Python per il calcolo scientifico con Numpy e Matplotlib



Introduzione

La prima esercitazione è rivolta a una introduzione pratica a Python per il calcolo scientifico, con particolare enfasi su due librerie fondamentali: Numpy e Matplotlib.

- **Numpy** è una libreria essenziale per il calcolo scientifico in Python, offrendo supporto per array multidimensionali, funzioni matematiche avanzate e molto altro ancora.
- **Matplotlib** è una libreria per la visualizzazione dei dati, che consente di creare grafici e grafici di alta qualità.

Questi esercizi semplici sono scelti per aiutare ad acquisire familiarità con i concetti e le funzionalità presentate nell'esercitazione. Gli esercizi coprono una vasta gamma di argomenti, inclusi l'uso di array, l'applicazione di funzioni matematiche, la manipolazione dei dati e la creazione di grafici utilizzando Matplotlib.



Riassunto Esercitazione 1

In questa esercitazione si è introdotto l'uso di **Python per il calcolo scientifico**, con particolare attenzione a due librerie fondamentali:

- **Numpy**: gestione di array multidimensionali, funzioni matematiche e operazioni algebriche di base e avanzate.
- **Matplotlib**: visualizzazione dei dati e creazione di grafici.

Gli esercizi hanno guidato passo dopo passo attraverso concetti essenziali, tra cui:

- creazione e manipolazione di array e vettori;
- calcolo di grandezze statistiche e valori caratteristici (minimo, massimo, somme);
- definizione di funzioni personalizzate (es. prodotto scalare, norme ℓ_p);
- verifica di proprietà matematiche (identità del parallelogramma per la norma euclidea);
- operazioni su matrici (somma, differenza, prodotto puntuale, prodotto di Kronecker, applicazioni a vettori);
- rappresentazione grafica di funzioni e segnali;
- implementazione di funzioni a tratti e loro visualizzazione.

L'obiettivo principale è stato **acquisire familiarità con gli strumenti fondamentali di Numpy e Matplotlib** attraverso esempi pratici, gettando le basi per analisi numerica e calcolo scientifico in Python.



Introduzione al Calcolo Scientifico con Python

Si introduce Python come strumento per il **calcolo scientifico**, con particolare attenzione a **Numpy e Matplotlib**.

Python

- Linguaggio di programmazione ad alto livello, semplice e con sintassi pulita.
- **Interpretato**, multiplatforma, open-source, supportato da una vasta comunità.
- Utilizzato anche in ambito industriale (Google, ecc.).
- **Anaconda** è una distribuzione consigliata per il calcolo scientifico (include Numpy, Scipy, Matplotlib e IDE Spyder).

Strutture e sintassi di base

- **Indentazione** fondamentale al posto delle parentesi graffe `{}`.
- **Tipi di variabili non dichiarati**: dedotti automaticamente.
- **Operatori logici** testuali: `and`, `or`, `not`.
- Funzioni definite con `def`, salvate in file `.py` che costituiscono moduli.
- Importazioni flessibili (`import`, `from ... import ...`).
- Strutture di controllo: `if`, `for`, `while`.

Numpy

- Libreria per la gestione di **array multidimensionali** e calcolo numerico.
- **Array** (`ndarray`): omogenei, con attributi principali `shape`, `size`, `ndim`, `dtype`.
- **Funzioni per la creazione**:
 - `np.array`, `np.zeros`, `np.ones`, `np.arange`, `np.linspace`, `np.logspace`.
 - Matrici: `np.identity`, `np.eye`, `np.diag`.
- **Indicizzazione e slicing**: accesso con `[]`, supporta slicing avanzati, viste (`views`) e copie (`copy`).
- **Reshape e resize**: `reshape`, `flatten`, `ravel`, `squeeze`, `transpose`, `concatenate`.
- **Operazioni elementwise**: supporto a operatori (`+`, `-`, `/`, `*`, ...) e funzioni matematiche (`np.sin`, `np.exp`, `np.sqrt`, ...).
- **Broadcasting**: adattamento automatico di array di forme diverse.
- **Funzioni aggregate**: `np.mean`, `np.sum`, `np.std`, `np.min`, `np.max`, `np.argmax`, `np.argmin`, `np.all`, `np.any`.
- **Indicizzazione avanzata**: fancy indexing e maschere booleane.
- **Funzioni logiche**: `np.where`, `np.logical_and`, `np.logical_or`, ecc.
- **Operazioni matriciali**:
 - `np.dot` (prodotto scalare/matrice),
 - `np.inner`, `np.cross`, `np.outer`, `np.tensordot`, `np.kron`, `np.einsum`.

Matplotlib

- Libreria per la **visualizzazione scientifica**.
- Creazione di grafici 2D, figure personalizzabili, supporto a stili e annotazioni.

In sintesi, si introducono le **basi di Python**, l'uso di **Numpy** per array e calcoli numerici, e l'importanza di **Matplotlib** per la rappresentazione grafica, fornendo gli strumenti essenziali per il calcolo scientifico e l'analisi dei dati in Python.

```
In [6]: import numpy as np
import matplotlib.pyplot as plt
from numpy import linalg as la
```

🚀 Esercizio 1

Generare il vettore riga **a** e il vettore colonna **b** di interi da 1 a 10 e da 10 a 1, rispettivamente. Generare poi il vettore riga **c** contenente 11 nodi equispaziati nell'intervallo [0, 1].

```
In [7]: a=np.arange(1,11,1)
print(f"a = {a}")
print(f"Numero elementi per ciascuna dimensione è {a.shape[0]}\n")

b=np.column_stack(a[::-1])
print(f"b = {b}")
print(f"Numero elementi per ciascuna dimensione è {b.shape}\n")

c=np.linspace(0,1,11)
print(f"c = {c}")
print(f"Numero elementi del vettore è {c.size}")
```

```
a = [ 1  2  3  4  5  6  7  8  9 10]
Numero elementi per ciascuna dimensione è 10
```

```
b = [[10  9  8  7  6  5  4  3  2  1]]
Numero elementi per ciascuna dimensione è (1, 10)
```

```
c = [0.  0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9 1. ]
Numero elementi del vettore è 11
```

🚀 Esercizio 2

Generare un vettore di numeri casuali distribuiti uniformemente in $[-1, 1]$. Determinare il valore massimo, minimo, il valore assoluto massimo, quello minimo, la somma degli elementi e la somma dei valori assoluti degli elementi.

```
In [8]: data=2*np.random.rand(10)-1

print(f'data = {data}\n')

print(f"L'elemento massimo è {np.max(data)}")
print(f"L'elemento minimo è {np.min(data)}")
print(f"Il valore assoluto massimo è {np.max(np.abs(data))}")
print(f"Il valore assoluto minimo è {np.min(np.abs(data))}")
print(f"La somma degli elementi è {np.sum(data)}")
print(f"La somma dei valori assoluti è {np.sum(np.abs(data))}")

data = [-0.60268652  0.36712182 -0.45303899  0.3940959   0.85131978  0.79653761
 -0.21607175  0.61743716  0.34340526 -0.80606302]
```

```
L'elemento massimo è 0.8513197793333884
L'elemento minimo è -0.8060630178332069
Il valore assoluto massimo è 0.8513197793333884
Il valore assoluto minimo è 0.2160717537963872
La somma degli elementi è 1.2920572600360547
La somma dei valori assoluti è 5.447777821094606
```

🚀 Esercizio 3

Scrivere una funzione `ps` in Python che prenda in input due vettori **a**, **b** e dia in output il prodotto scalare dei vettori. Inserire un controllo preliminare che verifichi se è possibile calcolare il prodotto scalare e dia un messaggio di errore se la lunghezza dei vettori non è compatibile. Considerare anche il vettore con entrate complesse.

```
In [9]: def ps(x, y):
    x = np.asarray(x)
    y = np.asarray(y)

    if x.shape != y.shape or x.ndim != 1:
        raise ValueError("I vettori devono avere la stessa lunghezza e una sola dimensione.")

    return np.inner(x, np.conj(y))

# ✅ Esempio 1: Vettori reali
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(f"Prodotto scalare (reale): {ps(a, b)}") # Output: 1*4 + 2*5 + 3*6 = 32

# ✅ Esempio 2: Vettori complessi
c = np.array([1+2j, 3+4j])
d = np.array([5+6j, 7+8j])
print(f"Prodotto scalare (complesso): {ps(c, d)}")
```

```
# Calcolo manuale:
# (1+2j) * conj(5+6j) = (1+2j)*(5-6j) = 17 + 4j
# (3+4j) * conj(7+8j) = (3+4j)*(7-8j) = 53 + 4j
# Somma: (17 + 4j) + (53 + 4j) = 70 + 8j

# ✖ Esempio 3: Errore dimensione
try:
    ps([1, 2], [1, 2, 3])
except ValueError as e:
    print(f"Errore: {e}")
```

Prodotto scalare (reale): 32

Prodotto scalare (complesso): (70+8j)

Errore: I vettori devono avere la stessa lunghezza e una sola dimensione.

✚ Esercizio 4

Siano

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}, \quad B = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}, \quad b = \begin{pmatrix} 3 \\ 5 \\ 7 \end{pmatrix}.$$

Calcolare:

- $A + A$
- $A - B$
- $A \odot A$ (prodotto puntuale elemento per elemento)
- A^2
- $A \otimes B$ (prodotto di Kronecker)
- $B \otimes A$
- Ab
- $b^T A$

```
In [10]: aux=np.arange(1,10,1)
A=np.reshape(aux,(3,3))
B=np.eye(3)

print(f"A = \n {A}")

b=np.array([[3],[5],[7]])
print(f"b = \n {b}")

print(f"Somma = \n {A + A}")
print(f"Differenza = \n {A - B}")
print(f"Prodotto puntuale = \n {A * A}")
print(f"Prodotto matriciale = \n {np.dot(A, A)}")
```

```
A =
[[1 2 3]
 [4 5 6]
 [7 8 9]]
b =
[[3]
 [5]
 [7]]
Somma =
[[ 2  4  6]
 [ 8 10 12]
 [14 16 18]]
Differenza =
[[0.  2.  3.]
 [4.  4.  6.]
 [7.  8.  8.]]
Prodotto puntuale =
[[ 1  4  9]
 [16 25 36]
 [49 64 81]]
Prodotto matriciale =
[[ 30  36  42]
 [ 66  81  96]
 [102 126 150]]
```

```
In [11]: print(f"kron(A,B) =\n{np.kron(A, B)}")
print(f"kron(B,A) =\n{np.kron(B, A)}")
print(f"Ab = \n {np.dot(A, b)}")
print(f"b^T A = \n {np.dot(b.T, A)}")
```

```

kron(A,B) =
[[1. 0. 0. 2. 0. 0. 3. 0. 0.]
 [0. 1. 0. 0. 2. 0. 0. 3. 0.]
 [0. 0. 1. 0. 0. 2. 0. 0. 3.]
 [4. 0. 0. 5. 0. 0. 6. 0. 0.]
 [0. 4. 0. 0. 5. 0. 0. 6. 0.]
 [0. 0. 4. 0. 0. 5. 0. 0. 6.]
 [7. 0. 0. 8. 0. 0. 9. 0. 0.]
 [0. 7. 0. 0. 8. 0. 0. 9. 0.]
 [0. 0. 7. 0. 0. 8. 0. 0. 9.]]
kron(B,A) =
[[1. 2. 3. 0. 0. 0. 0. 0. 0.]
 [4. 5. 6. 0. 0. 0. 0. 0. 0.]
 [7. 8. 9. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 1. 2. 3. 0. 0. 0.]
 [0. 0. 0. 4. 5. 6. 0. 0. 0.]
 [0. 0. 0. 7. 8. 9. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 1. 2. 3.]
 [0. 0. 0. 0. 0. 0. 4. 5. 6.]
 [0. 0. 0. 0. 0. 0. 7. 8. 9.]]
Ab =
[[ 34]
 [ 79]
 [124]]
b^T A =
[[ 72  87 102]]

```

🚀 Esercizio 5

Scrivere una funzione che abbia in entrata un vettore $\mathbf{v}$ ed un numero reale $p \geq 1$ e dia in uscita la norma ℓ_p del vettore, ovvero, se $v = [v_1, v_2, \dots, v_N]^T$

$$\|v\|_p = \left(\sum_{i=1}^N |v_i|^p \right)^{1/p}$$

Calcolare la norma dei vettori introdotti nel punto 1).

```

In [12]: def norm(x, p, flag):
          x = np.asarray(x)

          if x.ndim != 1:
              raise ValueError("Input non valido: x deve essere un vettore 1D.")
          if p < 1:
              raise ValueError("La norma ℓ_p è definita solo per p ≥ 1.")

          if flag == 1:
              return la.norm(x, p)
          elif flag == 0:
              return np.power(np.sum(np.abs(x)**p), 1/p)
          else:
              raise ValueError("Flag deve essere 0 o 1.")

          print(f"Norma L1 di a con la: {norm(a, 1, 0)}")
          print(f"Norma L1 di a senza la: {norm(a, 1, 1)}\n")

          print(f"Norma L2 di c con la: {norm(c, 2, 0)}")
          print(f"Norma L2 di c senza la: {norm(c, 2, 1)}")

```

Norma L1 di a con la: 6.0

Norma L1 di a senza la: 6.0

Norma L2 di c con la: 5.477225575051661

Norma L2 di c senza la: 5.477225575051661

🚀 Esercizio 6

Utilizzando la funzione precedentemente definita, verificare che con $p = 2$ vale l'identità del parallelogramma mentre con altri valori di p non è verificata. Commentare il risultato.

Identità del parallelogramma

L'identità del parallelogramma, valida solo per la norma euclidea (ℓ_2), afferma che:

$$\|v_1 + v_2\|^2 + \|v_1 - v_2\|^2 = 2 (\|v_1\|^2 + \|v_2\|^2)$$

Questa identità vale solo per la norma ℓ_2 , cioè per $p = 2$.

```

In [13]: def id_par(p):
          v_1=np.random.rand(10)
          v_2=np.random.rand(10)
          if np.abs(norm(v_1+v_2,p,1)**2+norm(v_1-v_2,p,1)**2-2*(norm(v_1,p,1)**2+norm(v_2,p,1)**2))<10**(-10):
              print(f"Identità del parallelogramma è valida per p = {p}")
              return
          else:
              print(f"Identità del parallelogramma non è valida per p = {p}")
              return

          for p in [1, 1.5, 2, 3, 10]:
              id_par(p)

```

Identità del parallelogramma non è valida per $p = 1$
Identità del parallelogramma non è valida per $p = 1.5$
Identità del parallelogramma è valida per $p = 2$
Identità del parallelogramma non è valida per $p = 3$
Identità del parallelogramma non è valida per $p = 10$

🚀 Esercizio 7

Rappresentare graficamente i seguenti segnali e calcolarne il prodotto scalare:

- $a = \sin(2\pi t)$, $b = t$;
- $a = \sin(6\pi t)$, $b = \sin(4\pi t)$;
- $a = \cos\left(\frac{3}{2}\pi t\right)$, $b = t$;
- $a = b = e^{4\pi i t}$;

dove $\mathbf{t}$ è un vettore equispaziato di 100 punti compresi tra -1 e 1.

```
In [14]: # Vettore t
t = np.linspace(-1, 1, 100)

# ---- Caso 1 ----
a = np.sin(2 * np.pi * t)
b = t
fig, ax = plt.subplots()
plt.plot(t, a, 'r--', label="sin(2πt)")
plt.plot(t, b, 'b--', label="t")
plt.title('a = sin(2πt), b = t')
ax.set_xlabel("t")
ax.set_ylabel("y")
ax.legend()
plt.grid(True)
plt.show()

res1 = ps(a, b)
print(f"Il risultato è {res1}")

# ---- Caso 2 ----
a = np.sin(6 * np.pi * t)
b = np.sin(4 * np.pi * t)
fig, ax = plt.subplots()
plt.plot(t, a, 'r--', label="sin(6πt)")
plt.plot(t, b, 'b--', label="sin(4πt)")
plt.title('a = sin(6πt), b = sin(4πt)')
ax.set_xlabel("t")
ax.set_ylabel("y")
ax.legend()
plt.grid(True)
plt.show()

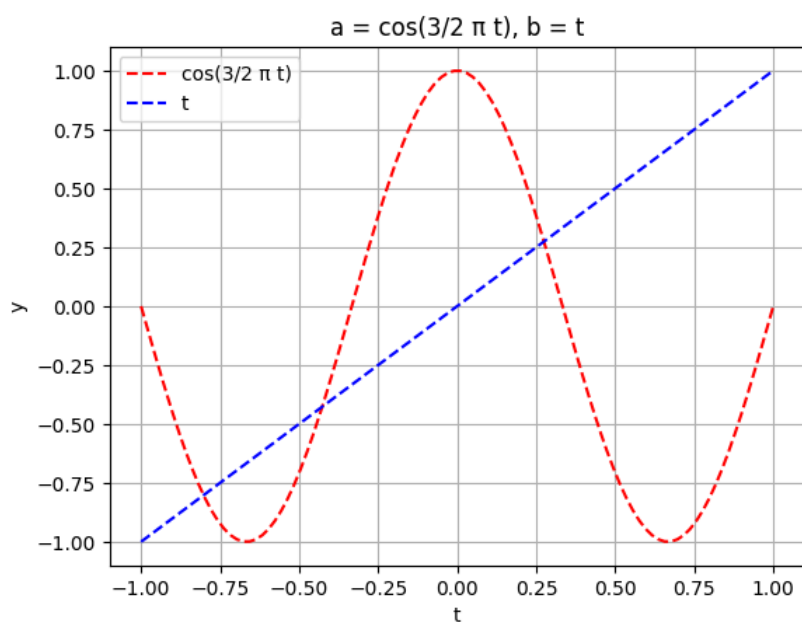
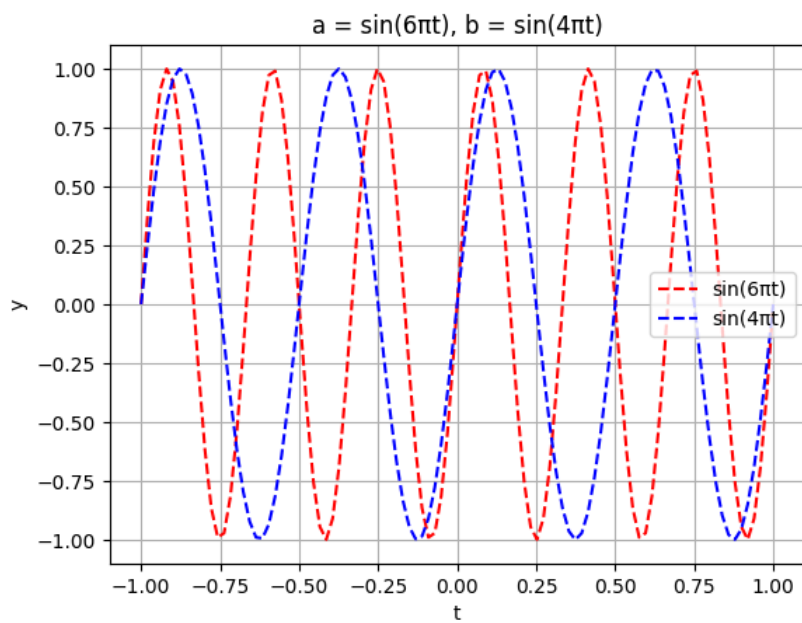
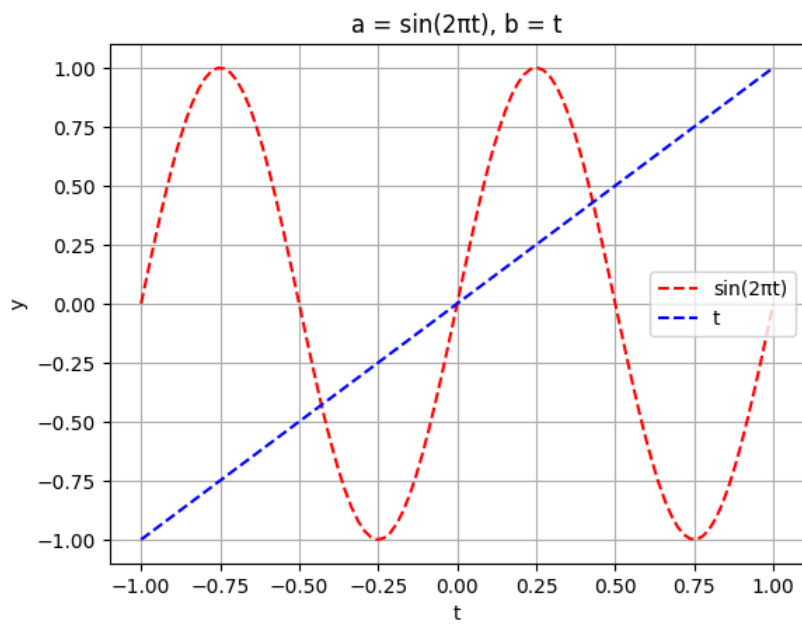
res2 = ps(a, b)
print(f"Il risultato è {res2}")

# ---- Caso 3 ----
c = 3/2
a = np.cos(c * np.pi * t)
b = t
fig, ax = plt.subplots()
plt.plot(t, a, 'r--', label="cos(3/2 π t)")
plt.plot(t, b, 'b--', label="t")
plt.title('a = cos(3/2 π t), b = t')
ax.set_xlabel("t")
ax.set_ylabel("y")
ax.legend()
plt.grid(True)
plt.show()

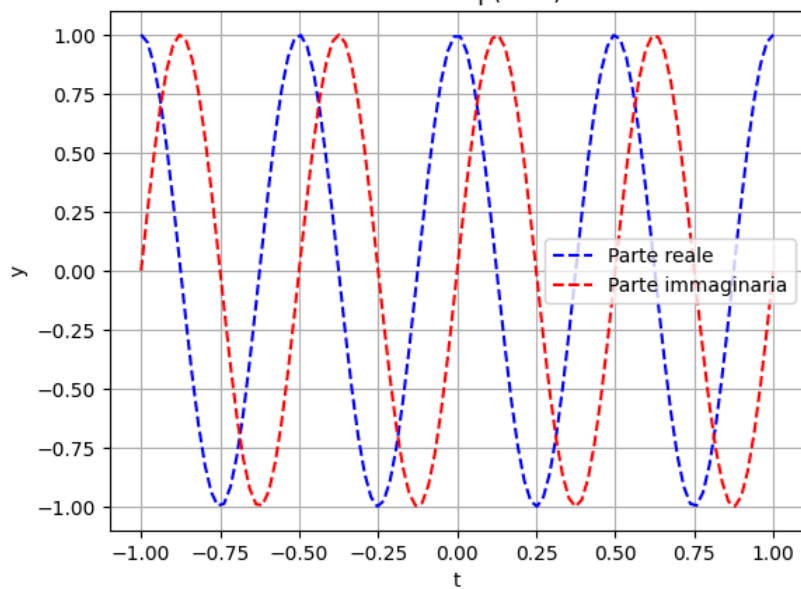
res3 = ps(a, b)
print(f"Il risultato è {res3}")

# ---- Caso 4 ----
a = np.exp(4 * np.pi * 1j * t)
b = a
fig, ax = plt.subplots()
plt.plot(t, np.real(a), 'b--', label="Parte reale")
plt.plot(t, np.imag(a), 'r--', label="Parte immaginaria")
plt.title('a = b = exp(4πi t)')
ax.set_xlabel("t")
ax.set_ylabel("y")
ax.legend()
plt.grid(True)
plt.show()

res4 = ps(a, b)
print(f"Il risultato è {res4}")
```



$$a = b = \exp(4\pi i t)$$



Il risultato è $(100+0j)$

✚ Esercizio 8

Scrivere una funzione che abbia come input un vettore $\mathbf{x}$ e gli scalari x_0 , A e w e restituisca la funzione:

$$y(x) = \begin{cases} A & \text{se } x \in [x_0, x_0 + w] \\ 0 & \text{altrimenti} \end{cases}$$

Usando la funzione definita, disegnare la funzione nell'intervallo $[-5, 5]$ con $A = 1$, $x_0 = -2$, $w = 3$.

```
In [15]: def pulse(x, position, width, height):
    return height*np.logical_and(x>=position, x<=(position+width))

# Parametri specificati nell'esercizio
A = 1
x0 = -2
w = 3
x = np.linspace(-5, 5, 1000)

# Calcolo impulso
y = pulse(x, x0, w, A)

# Plot
plt.figure(figsize=(8, 4))
plt.plot(x, y, 'b-', linewidth=2)
plt.title(r'$y(x) = A$ su $[x_0, x_0 + w]$, altrimenti 0' + f"\n(A={A}, x0={x0}, w={w})")
plt.xlabel("x")
plt.ylabel("y")
plt.grid(True)
plt.ylim(-0.1, A + 0.5)
plt.show()
```

$$y(x) = A \text{ su } [x_0, x_0 + w], \text{ altrimenti } 0$$

(A=1, x₀=-2, w=3)

